Mahidol University
Wisdom of the Land

Mahidol University



# ITDS241 Web Technologies and Applications

## Introduction to JavaScript

Instructor: Asst. Prof. Wudhichart Sawangphol, Ph.D., SFHEA
Email: wudhichart.saw@mahidol.ac.th

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Class Objectives



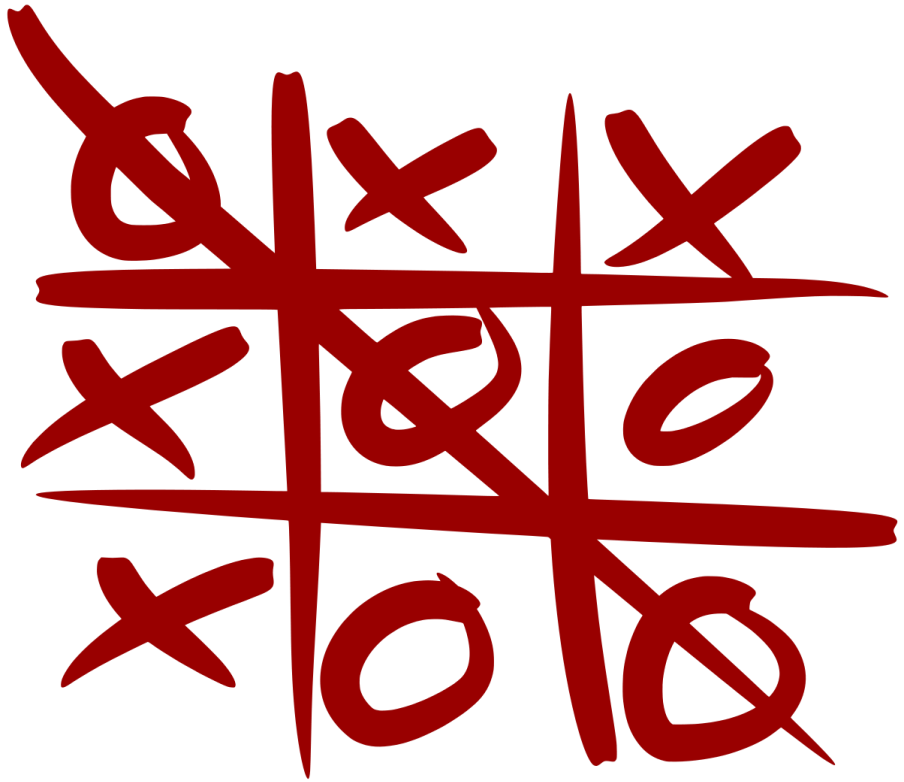
- Students will be able
  - To explain the basic concept of JavaScript
  - To write basic JavaScript
  - To explain and apply the execution control structures

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Tic Tac Toe



- We are going to create the Tic Tac Toe game using JavaScript, HTML, and CSS



Ref: https://en.wikipedia.org/wiki/Tic-tac-toe#/media/File:Tic_tac_toe.svg

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Tic Tac Toe: How it works ?



- Let’s start by UI
  - Title
  - 3x3 grid
    - Each cell can be clicked
    - The sign of player is displayed on each cell
  - The message to show the player’s turn
  - The message to show who won the game or draw
  - Restart button

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Tic Tac Toe: How it works ? (cont.)



- Game Flow
  - Track any clicks on cells
  - Check if the move is valid
  - Should update our game state
  - Check the game state, who is won, or the game is draw
  - Stop the game when the conditions met
  - Update UI
  - Restart

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# HTML for Tic Tac Toe



```
<html lang="en">
<head>
<title>Tic Tac Toe</title>
<link rel="stylesheet" href="style.css" />
</head>
<body>
<section>
<h1 class="game-title">Tic Tac Toe</h1>
<div class="game-container">
<div data-cell-index="0" class="cell"></div>
<div data-cell-index="1" class="cell"></div>
<div data-cell-index="2" class="cell"></div>
<div data-cell-index="3" class="cell"></div>
<div data-cell-index="4" class="cell"></div>
<div data-cell-index="5" class="cell"></div>
<div data-cell-index="6" class="cell"></div>
<div data-cell-index="7" class="cell"></div>
<div data-cell-index="8" class="cell"></div>
</div>
<h2 class="game-status"></h2>
<button class="game-restart">Restart Game</button>
</section>
</body>
<script src="script.js"></script>
</html>
```

- Use **`div`** for game container and cell
  - Title
  - 3x3 grid
    - Each cell can be clicked
    - The sign of player is displayed on each cell
  - The message to show the player's turn
  - The message to show who won the game or draw
  - Restart button

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# CSS for HTML

```
body {
font-family: "Arial", sans-serif;
}
section {
text-align: center;
}
.game-container {
display: grid;
grid-template-columns: repeat(3, auto);
width: 306px;
margin: 50px auto;
}
.cell {
font-family: "Permanent Marker", cursive;
width: 100px;
height: 100px;
box-shadow: 0 0 0 1px #333333;
border: 1px solid #333333;
cursor: pointer;
line-height: 100px;
font-size: 60px;
}
```

```
<html lang="en">
<head>
<title>Tic Tac Toe</title>
<link rel="stylesheet" href="style.css" />
</head>
<body>
<section>
<h1 class="game-title">Tic Tac Toe</h1>
<div class="game-container">
<div data-cell-index="0" class="cell"></div>
<div data-cell-index="1" class="cell"></div>
<div data-cell-index="2" class="cell"></div>
<div data-cell-index="3" class="cell"></div>
<div data-cell-index="4" class="cell"></div>
<div data-cell-index="5" class="cell"></div>
<div data-cell-index="6" class="cell"></div>
<div data-cell-index="7" class="cell"></div>
<div data-cell-index="8" class="cell"></div>
</div>
<h2 class="game-status"></h2>
<button class="game-restart">Restart Game</button>
</section>
</body>
<script src="script.js"></script>
</html>
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS for Tic Tac Toe



```
<html lang="en">
<head>
<title>Tic Tac Toe</title>
<link rel="stylesheet" href="style.css" />
</head>
<body>
<section>
<h1 class="game-title">Tic Tac Toe</h1>
<div class="game-container">
<div data-cell-index="0" class="cell"></div>
<div data-cell-index="1" class="cell"></div>
<div data-cell-index="2" class="cell"></div>
<div data-cell-index="3" class="cell"></div>
<div data-cell-index="4" class="cell"></div>
<div data-cell-index="5" class="cell"></div>
<div data-cell-index="6" class="cell"></div>
<div data-cell-index="7" class="cell"></div>
<div data-cell-index="8" class="cell"></div>
</div>
<h2 class="game-status"></h2>
<button class="game-restart">Restart Game</button>
</section>
</body>
<script src="script.js"></script>
</html>
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS for Tic Tac Toe (cont.)



```
/*
Add Event listener to cell
*/
document
.querySelectorAll(".cell")
.forEach((cell) => cell.addEventListener("click",
handleCellClick));
```

- Use **`div`** for game container and cell
  - Title
  - 3x3 grid
    - Each cell can be clicked
    - The sign of player is displayed on each cell
  - The message to show the player's turn
  - The message to show who won the game or draw
  - Restart button

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS for Tic Tac Toe (cont.)



```
function handleCellPlayed() {}

function handlePlayerChange() {}

function handleResultValidation() {}

function handleCellClick() {}

function handleRestartGame() {}

document
.querySelector(".game-restart")
.addEventListener("click", handleRestartGame);
```

- Track any clicks on cells
- Check if the move is valid
- Should update our game state
- Check the game state, who is won, or the game is draw
- Stop the game when the conditions met
- Update UI
- Restart

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS for Tic Tac Toe (cont.)



```
function handleCellPlayed() {}

function handlePlayerChange() {}

function handleResultValidation() {}

function handleCellClick() {}

function handleRestartGame() {}

document
.querySelector(".game-restart")
.addEventListener("click", handleRestartGame);
```

```
function handleCellPlayed(clickedCell, clickedCellIndex)
{
/*
Update the game state to reflect the played move,
as well as update the user interface to reflect the
played move
*/
gameState[clickedCellIndex] = currentPlayer;
clickedCell.innerHTML = currentPlayer;
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS for Tic Tac Toe (cont.)



```
function handleCellPlayed() {}

function handlePlayerChange() {}

function handleResultValidation() {}

function handleCellClick() {}

function handleRestartGame() {}

document
.querySelector(".game-restart")
.addEventListener("click", handleRestartGame);
```

```
function handlePlayerChange() {
currentPlayer = currentPlayer === "X" ? "O" : "X";
statusDisplay.innerHTML = currentPlayerTurn();
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS for Tic Tac Toe (cont.)

```
function handleCellPlayed() {}

function handlePlayerChange() {}

function handleResultValidation() {}

function handleCellClick() {}

function handleRestartGame() {}

document
.querySelector(".game-restart")
.addEventListener("click", handleRestartGame);
```

```
const winningConditions = [
[0, 1, 2],
[3, 4, 5],
[6, 7, 8],
[0, 3, 6],
[1, 4, 7],
[2, 5, 8],
[0, 4, 8],
[2, 4, 6],
];

function handleResultValidation() {
let roundWon = false;
for (let i = 0; i <= winningConditions.length - 1; i++) {
const winCondition = winningConditions[i];
let a = gameState[winCondition[0]];
let b = gameState[winCondition[1]];
let c = gameState[winCondition[2]];
if (a === "" || b === "" || c === "") {
continue;
}
if (a === b && b === c) {
roundWon = true;
break;
}
}
if (roundWon) {
statusDisplay.innerHTML = winningMessage();
gameActive = false;
return;
}
/*
Check weather there are any empty cells from gameState
*/
let roundDraw = !gameState.includes("");
if (roundDraw) {
statusDisplay.innerHTML = drawMessage();
gameActive = false;
return;
}
/*
If it reach here, there are still moves to be played.
*/
handlePlayerChange();
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS for Tic Tac Toe (cont.)

```
function handleCellPlayed() {}

function handlePlayerChange() {}

function handleResultValidation() {}

function handleCellClick() {}

function handleRestartGame() {}

document
.querySelector(".game-restart")
.addEventListener("click", handleRestartGame);
```

```
function handleCellClick(clickedCellEvent) {
/*
Refer to the clicked cell
*/
const clickedCell = clickedCellEvent.target;
/*
Get the index of clicked cell from the attribute 'data-
cell-index'. The return value is string.
*/
const clickedCellIndex = parseInt(
clickedCell.getAttribute("data-cell-index")
);
/*
Check whether the cell has already been played,
or if the game is paused. If either of those is true we
will simply ignore the click.
*/
if (gameState[clickedCellIndex] !== "" || !gameActive) {
return;
}
/*
If everything is in the order we will proceed with the
game flow
*/
handleCellPlayed(clickedCell, clickedCellIndex);
handleResultValidation();
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS for Tic Tac Toe (cont.)



```
function handleCellPlayed() {}

function handlePlayerChange() {}

function handleResultValidation() {}

function handleCellClick() {}

function handleRestartGame() {}

document
.querySelector(".game-restart")
.addEventListener("click", handleRestartGame);
```

```
function handleRestartGame() {
gameActive = true;
currentPlayer = "X";
gameState = ["", "", "", "", "", "", "", "", ""];
statusDisplay.innerHTML = currentPlayerTurn();
document.querySelectorAll(".cell").forEach((cell)
=> (cell.innerHTML = ""));
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Contents

- What is JavaScript?
- How to use JavaScript?
- JavaScript Variables
- JavaScript Data Types and Operators
- Execution Control Structures

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.



Mahidol University
Wisdom of the Land

Mahidol University

# What is JavaScript?

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

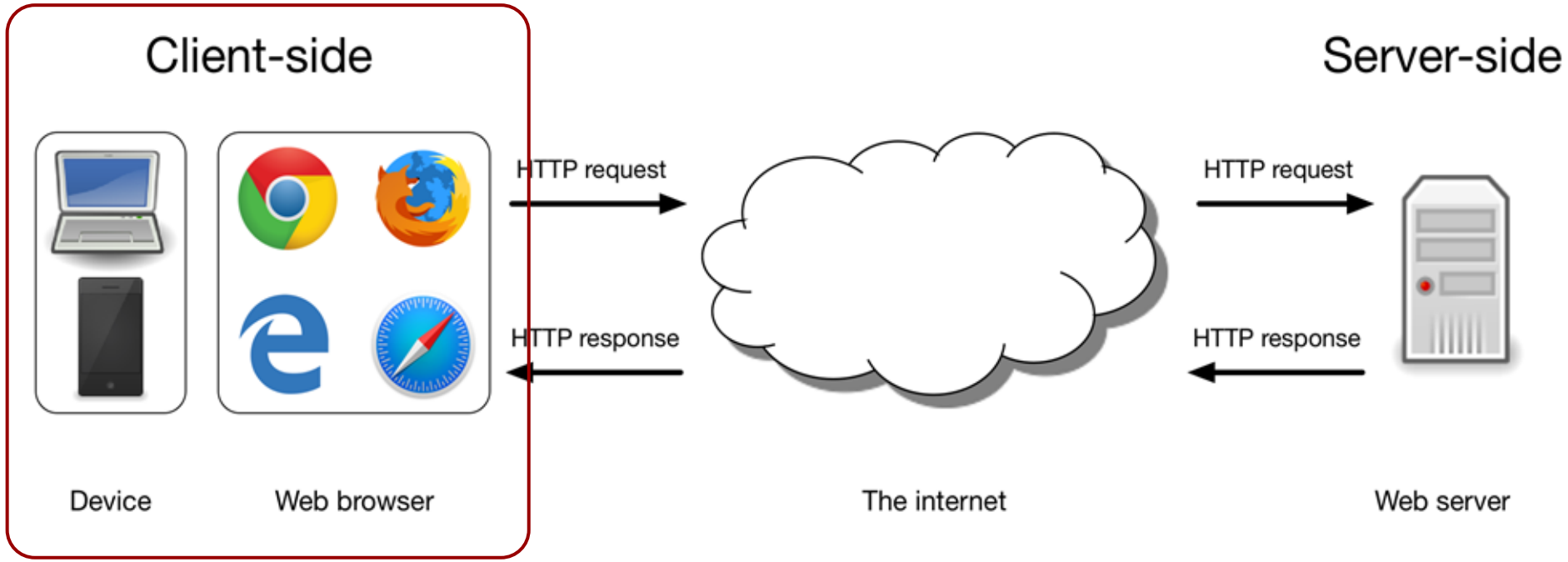
# Web Application Components



- A single **HTML file** describing the content and structure of the page
- Zero or more **media files**, e.g., images, audio or video
- Zero or more **CSS files** that describe the presentation or style of page elements
- Zero or more **JavaScript files** that specify dynamic, interactive behaviour

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# How does a Web Page Work?




COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# What is JavaScript?



- JS was designed to add **interactivity** to HTML pages
- JS is a scripting language, a **lightweight** programming language
- JS is usually **embedded directly** into HTML pages
- JS is an **interpreted** language (means that scripts execute without preliminary compilation)
- JS is accompanied by a vast array of prewritten libraries of code called APIs (Application Programming Interfaces), see https://developer.mozilla.org/en/docs/Web/API.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# What is JavaScript? (cont.)



- JS can put **dynamic text** into an HTML page
- JS can **react to events**
- JS can **read and write HTML elements**
- JS can be used to **validate data**
- JS can be used to **detect the visitor's browser**
- JS can be used to **create cookies**
- Why JavaScript?

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.



Mahidol University
Wisdom of the Land

Mahidol University

# How to use JavaScript?

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# How to use JavaScript?



- We can **insert** JavaScript in **between parts of the HTML code** using

something.html

```
<script>    // Starting of JavaScript
      /* --- This is your JS Code --- */
      console.log("Hello World!!!");
   /* ---------------------------- */
</script> // End of JavaScript
```

- Script tag can be inserted anywhere in `<head>`, `<body>`, or both

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# How to use JavaScript?



We can also import JavaScript from **`.js` file** using

something.html

```
<script src="myScript.js"></script>
```

myScript.js

```
/* All JS codes are written here */
console.log("Hello Word!!!");
```

**Note**: No need to put `<script>` in the `.js` file. All syntax is in JS

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.



Mahidol University
Wisdom of the Land

Mahidol University

# JS Variables

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JavaScript Variables



- JS Variables
- Variable declaration
- Assignment statement
- Displaying Output

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Example: JavaScript



```
<script>
        var firstName = "Wudhichart"; // Declaration and Initialization
        var lastName, fullinfo; // Declaration
        var ID;

        ID = "9999999" //Assignment
        lastName = "Sawangphol";

        /* String concatenation */
        fullinfo = ID + " " + firstName + " " + lastName;

        /* Output to the Console */
        console.log(fullinfo);
</script>
```

Output

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Variables

- **Variable**: a way for a program to temporarily store a piece of information and subsequently refer it it by a name

```
var firstName = "Wudhichart";
var lastName;
```

- The keyword **`var`** indicates a variable to be declared.
- Declaring a variable requires a **variable name** and/or an **initial value** for the variable.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Variable Name Rules



- They **must start** with **a letter**
- Containing only **a mix of letters and digits**, no spaces.
- Variable names **cannot be a JavaScript keyword** (e.g., `var`).

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Assignment statement



Unlike mathematics, = does not represent an equality sign in JavaScript. It is instead an **assignment** operator

```
var firstName = "Wudhichart"; //Declaration & Initialization
firstName = "Jidapa";  // Assignment

let nickName;          // Declaration
nickName = "Pa"; // Assignment
firstName = firstName + " " + nickName  // Assignment
console.log(firstName);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Variable declarations (cont.)



Regarding ES6, a variable in JavaScript can be declared using **`let`** and **`const`**

**`let`** is used to reduce redeclaration problem in code block

**`let`** can be updated but not re-declared.

```
var number = 1; /* First code block */
{       /* Another code block */
    var number = 4;
}
console.log("number is " + number);
```

→ `number is ?`

```
let number = 2; /* First code block */
{       /* Another code block */
    let number = 5;
}
console.log("number is " + number);
```

→ `number is ?`

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Variable declarations (cont.)



- More examples

```
var i = 5;
for (var i = 0; i<10;i++){
   // some statements
}
console.log("i is " + i);
```

```
let j = 5;
for (let j = 0; j<10;i++){
   // some statements
}
console.log("j is " + i);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Variable declarations (cont.)



- **`const`** is used to declare a variable with **constant values**.

```
const number = 1;
```

- **`const`** cannot be updated or re-declared

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Variable declarations (cont.)




COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Displaying Output



- There are two to display an output:
  - On console (Mainly used for developers)
  - On webpage

```
console.log(fullinfo);
```

- **`console.log()`** instructs JavaScript to log (write out) to the console the value of the expression

- To open the console:
  - Windows: F12
  - macOS: option+cmd+J

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# --- Check understanding ---



- Create an HTML page with JS:
  - Declare a variable named `number1`
  - Output `number1` (in the console)
  - Assign `number1` with 10
  - Output `number1`
  - Add 10 to the variable `number1`
  - Output `number1`

- **What is the output in your Console?**

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.



Mahidol University
Wisdom of the Land

Mahidol University

# JavaScript Data Types and Operators

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JavaScript Data Types



Six data types in JS

- **Boolean**: a logical entity and can have two values: `true` and `false`.
- **Number**: a numerical value range between $-(2^{53}-1)$ and $2^{53}-1$
- **String**: a textual data.
- **Null**: an intentional absence of any object value, i.e., “Nothing”
- **Undefined**: A variable that has not been assigned a value has the value undefined.
- **Objects** (User-defined and Built-in Objects)

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JavaScript Data Types



```
<script>
    //3. Data-types: Getting started with data types
    var outMessage = "";
    var myNum = 10;
    var myStr = "ten";
    var MyBool = true;

    outMessage = "The value of myNum is " + myNum + " and type is " + (typeof myNum);
    console.log(outMessage);

    outMessage = "The value of myStr is " + myStr + " and type is " + (typeof myStr);
    console.log(outMessage);

    outMessage = "The value of MyBool is " + MyBool + " and type is " + (typeof MyBool);
    console.log(outMessage);

    myNum = "Hello World!!"; // Is this Possible?

    outMessage = "The value of myNum is " + myNum + " and type is " + (typeof myNum);
    console.log(outMessage);
</script>
```

What is the output?

**JavaScript is weakly typed** holding different values of the **same** data type during program execution.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JavaScript Data Types: Objects



- User-defined object
- Built-in Standard objects
  - Number
  - Dates
  - Math
  - Array
  - String

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Objects: User-defined object



Objects can be described by many data items e.g., a person has a name, a gender, an age, etc.

```
<script>  //3. User-defined object

      var wudhichart = {
          firstName: "Wudhichart",
          lastName: "sawangphol",
          age: 30,
          feesPaid: true
      };

      var aCandidate, aProperty;
      aCandidate = wudhichart;

      wudhichart.age = 40;
      console.log("Wudhichart is " + wudhichart.age + " years old.");
      console.log("Candidate is " + aCandidate.age + " years old.");

      wudhichart["feesPaid"] = false;
      console.log("Candidate up-to-date with fees: " + aCandidate["feesPaid"]);

      aCandidate["parking permit"] = "blue";
      console.log("Wudhichart's parking permit is " + wudhichart["parking permit"]);

      aProperty = "parking permit";
      console.log("Wudhichart's parking permit is " + wudhichart[aProperty]);
      aProperty = "age";
      console.log("Wudhichart is " + wudhichart[aProperty] + " years old.");

</script>
```

What is the output?

Access each property by `.property` or `["property"]`

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Objects: Built-in Standard objects



- **Number**: a *wrapper* object allowing you to work with numerical values.
- **Math** : a built-in object that has properties and methods for mathematical including instance and function
- **Date**: a single moment in time. Date objects are based on a time value that is the number of milliseconds since 1 January 1970 UTC.
- **Array**:  a global object that is used in the construction of arrays; which are high-level, list-like objects.
- **String**: a constructor for strings, or a sequence of characters.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Built-in Standard objects: Number



- Create Number

```
var aNum1 = new Number("10");
var aNum2 = new Number("20");
```

- Methods, for examples,

```
Number.isNaN()
Number.isInteger()
Number.parseInt()
Number.parseFloat()
```

```
// NaN: Not a Number
console.log(Number.parseInt("10.569"));
console.log(Number.parseFloat("10.569"));
console.log(Number.parseInt("Hello"));
console.log(Number.isNaN(Number.parseInt("Hi")));
```

- Number instance methods, for examples,

```
numberObj.toExponential()
numberObj.toString()
numberObj.valueOf()
```

```
console.log(aNum1.toString());
console.log(aNum1.valueOf());
console.log(aNum1.toExponential());
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Built-in Standard objects: Math



- There is no *constructor* for Math

- Properties, for examples,
  - **Math.**PI
  - **Math.**SQRT2
- Methods, for examples,
  - **Math.**pow(x, y)
  - **Math.**round(x)
  - **Math.**abs(x)

```
console.log("The valus of PI is " + Math.PI);
console.log("5^2 = " + Math.pow(5,2));
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Built-in Standard objects: Date



- Create Date

```
var now = new Date(); // Current date and time
var aDate1 = new Date(value);  // Millisec
var aDate2 = new Date(dateString);
```

**Note:** `dateString` need to be in the format of ISO 8601, e.g., 2017-08-13T10:20:30 (**YYYY-MM-DDTHH:mm:ss**)

- Methods, for examples,

```
Date.parse()
```

- Date instance methods, for examples,

```
dateObj.getDate()
dateObj.getDay()
dateObj.getFullYear()
```

```
console.log(now.getDate());
console.log(now.getDay());
console.log(now.getFullYear());
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Built-in Standard Objects: Array



- Create array

```
var instructors = ["Wudhichart", "Wud"];
var anArray = new Array(5);
```

- Access array by index

```
console.log(instructors[0]);
```

- Array Properties

```
array.length // The number of elements in an array
```

- Methods

```
Array.isArray()
```

```
console.log(Array.isArray(instructors));
console.log(instructors.length);
console.log(instructors.indexOf("Wud"));
```

- Array instance methods

```
arrObj.pop()           //Remove and return the last element
arrObj.push()          //Adds one or more elements to the end
arrObj.indexOf(value)  //Returns the first index of an element
                       //that is equal to the specified value
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Built-in Standard Objects: Array (cont.)



- Array instance method
  - `arrobj.forEach(function) //executes a provided function once for each array element.`
  - `arrobj.map(function) //creates a new array with the results of calling a provided function on every element in the calling array`
  - `arrobj.filter()`
  - `arrobj.reduce()`
  - `arrobj.reduceRight()`
  - `arrobj.every()`
  - `arrobj.some()`
  - `arrobj.indexOf()`
  - `arrobj.lastIndexOf()`

Ref: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Built-in Standard Objects: String



- Create String

```
var str = "Hello World";                  //Primitive string
var str_obj = new String("Hello World"); // String object
```

- String concatenation by using **+**

- Properties

```
string.length
```

- String instance methods

```
stringObj.charAt()
stringObj.split()
stringObj.substring()
```

```
console.log(str_obj.length); // 11
str_obj.charAt(1);   // e
console.log(str_obj.split(" "));
console.log(str_obj.substring(3,10));
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JavaScript Operators



- Arithmetic operators
- Assignment operators
- Comparison operators
- Boolean (Logical) Operators
- Operator precedence

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS Operators: Arithmetic operators



- The basic **arithmetic** operators (`+`, `-`, `*`, `/`, and `%`) are binary operators
- JS provides the remainder operator , `%`, after division, e.g. , The result of `5%3` is 2

- **Parentheses** can be used to group expressions as in algebra.
- Operators in arithmetic expressions are applied in a precise sequence determined by the **rules of precedence** order
  - Multiplication, division and remainder operations are applied first.
  - Addition and subtraction operations are applied next.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS Operators: Arithmetic operators



## Summary of Arithmetic operators

| JavaScript Operators | Arithmetic Operators | JavaScript Expression |
|---|---|---|
| Addition | + | x + y |
| Subtraction | - | x - y |
| Division | / | x / y |
| Multiplication | * | x * y |
| Remainder | % | x % y |
| Exponentiation | ** | x ** y |
| Increment | ++ | ++x or x++ |
| Decrement | -- | --x or x-- |
| Unary negation | - | -x |
| Unary plus | + | +x |

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JavaScript Operators: Assignment operators



An **assignment operator** assigns a value of its right operand to its left operand.

| JavaScript Operators | Assignment Operators | JavaScript Expression | Meaning |
|---|---|---|---|
| Assignment | = | x = y | x = y |
| Addition Assignment | += | x += y | x = x + y |
| Subtraction Assignment | -= | x -= y | x = x - y |
| Division Assignment | /= | x /= y | x = x / y |
| Multiplication Assignment | *= | x *= y | x = x * y |
| Remainder Assignment | %= | x %= y | x = x % y |
| Exponentiation Assignment | **= | x **= y | x = x ** y |

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS Operators: Comparison operators



- JavaScript has both **strict** and **type–converting** comparisons.
  - A **strict** comparison is only true if the type and content of operands are the same (i.e., ===)

```
console.log("1" === 1);     // false
console.log("1" === "1");   // true
```

  - A **type–converting** comparison (commonly-used abstract comparison) convert the type of operands before comparing (i.e., ==)

```
console.log("1" == 1);      // true
console.log("1" == "1");    // true
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS Operators: Comparison operators



## Summary of Comparison operators

| JavaScript Operators | Comparison Operators | JavaScript Expression |
|---|---|---|
| Equality | == | x == y |
| Inequality | != | x != y |
| Strict equality | === | x === y |
| Strict inequality | !== | x !== y |
| Greater than | > | x > y |
| Greater than or equal | >= | x >= y |
| Less than | < | x < y |
| Less than or equal | <= | x <= y |

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JS Operators: Logical operators



**Logical operators** are normally used with **Boolean** value

| JavaScript Operators | Logical Operators | JavaScript Expression |
|---|---|---|
| Logical AND | && | expr1 && expr2 |
| Logical OR | \|\| | expr1 \|\| expr2 |
| Logical NOT | ! | ! expr |

| | | && | \|\| |
|---|---|---|---|
| T | T | T | T |
| T | F | F | T |
| F | T | F | T |
| F | F | F | F |

| | ! |
|---|---|
| T | F |
| F | T |

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# JavaScript Operators: Operator precedence



1. Grouping or the parentheses ()
2. Postfix increment (decrement)
3. Logical NOT, Unary Plus (Negation), Prefix increment (decrement) ordered, right-to-left
4. Exponentiation
5. Multiplication, Division, Remainder ordered,left-to-right
6. Addition, Subtraction ordered,left-to-right
7. Less Than (or Equal), Greater Than (or Equal), left-to-right
8. (Strict) Equality, (Strict) Inequality, left-to-right
9. Logical AND
10. Logical OR
11. Assignment

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.



Mahidol University
Wisdom of the Land

Mahidol University

# Execution Control Structure

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Execution Control Structure



- JS code has executed in a series of sequential statements
- JS has syntax that allows us to use **condition and a particular block of code is selected to execute** regarding the value of condition. This structure is called a **decision**.
- JS has syntax that allows a code block to be **repeatedly executed** while some condition is true. It's called a **repetition**.

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Execution Control Structures



- Decision control structures (**if-then-else**)
- Repetition control structures (**loops**)
  - Using Repetition control structure with Arrays and User-defined Object's Properties

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Decision control structures
# `if`



https://www.youtube.com/watch?v=5i-xUNp5zMQ

- **`if`** statement offers the **choice** of doing something or not doing it
- For example, suppose we define num1 = 1 and str1 = '1'

```
if(num1 === str1){
   console.log("The condition num1 === str1 is "
                              + (num1 === str1))
}
```

What is the output?

- The condition of **`if`** statement need to be logical or comparison operators. In addition, it can be something else that return Boolean value (e.g., functions)

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Decision control structures
# **if-else**

**if-else** statement

- use else with the choice to do something else when the condition of if is **false**.
- For example, suppose `num1=10` and `str1='1'`

```
if(num1 === str1){
   console.log("The condition num1 === str1 is " +
               (num1 === str1))
}
else{
   console.log("The condition num1 === str1 is " +
               (num1 === str1))
}
```

What is the output?

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Decision control structures
# Conditional (ternary) Operator



- This is operator is used as a shortcut for the **`if-else`** statement

***`condition ? expr1 : expr2`***

- Example

```
var num1 = 1;
var num2 = 3;
var biggerNumber = num1 > num2 ? num1 : num2;
console.log("biggerNumber is " + biggerNumber);
```

What is the output?

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Decision control structures
# `if-else-if`

**`if-else-if`** statement

- Nesting an inner **`if-else`** statement in the else part of the outer if-else statement
- For example,

```
var mark = 75;
var grade = "";

if ((mark >= 0) && (mark < 50)) {
    grade = "F";
} else if ((mark >= 50) && (mark < 60)) {
    grade = "D";
} else if (mark >= 60 && mark < 70) {
    grade = "C";
} else if (mark >= 70 && mark < 80) {
    grade = "B";
} else if (mark >= 80 && mark <= 100) {
    grade = "A";
} else {
    grade = "ERROR: range violation
                        for mark: " + mark;
}
console.log("The grade is " + grade);
```

What is the output?

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Decision control structures
# `switch`



## `switch` statement

- match expression's value to a case clause and then execute the code block associated with that case

```
switch (expression) {
case value1: //Statements executed when the result matches value1
    [break;]
case value2: //Statements executed when the result matches value2
    [break;]
...
case valueN: //Statements executed when the result matches valueN
    [break;]
[default: // Statements executed when none of the values match the
                        value of the expression [break;]]
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Decision control structures (cont.) `switch`



**`switch`** statement
For example,

What is the output?

```
var expr = "Soda";
switch (expr) {
    case 'Coke':
        console.log('Coke is 30 Baht.');
        break;
    case 'Diet Coke':
        console.log('Diet Coke is 30 Baht.');
        break;
    case 'Sprite':
        console.log('Sprite is 30 Baht.');
        break;
   case 'Ginger Ale':
        console.log('Ginger Ale is 30 Baht.');
        break;
    case 'Soda':
    case 'Sparking Water':
        console.log('Soda or Sparking Water is 30
        Baht.');
         break;
    default:
        console.log('Sorry, we are out of ' + expr +
    '.');
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Repetition Control Structures (loops)



https://www.youtube.com/watch?v=I15K6r_29rs

- Loop = iteration = Repetition

- Ensure that the loop condition is satisfied before the first iteration
- Ensure that the code block in the loop change the loop condition. Therefore, the loop can terminate, and infinite loops is avoided

- Types of loop
  - While loop
  - Do-While loop
  - For loop
  - Label

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Repetition Control Structures
# `while`



- The **`while`** statement allows us to have a loop that executes a code block repeatedly until the condition is evaluated to be false
- Syntax

```
while (condition){
  code block
}
```

```
var n = 0;
var x = 0;

while (n < 10) {
   n++;
   x += n;
}
console.log("n = "+n+", x = " + x);
```

What is the output?

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Repetition Control Structures
# do-while



- The **do-while** statement allows us to have a loop that executes a code block at least once and repeatedly executing the code block until the condition is evaluated to be false
- Syntax:

```
do{
      statement
}while(condition);
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Repetition Control Structures **do-while** (cont.)



```
// while vs. do-while
n = 1;
x = 0;
while (n < 1) {
    n++;
    x += n;
}
console.log("n = " + n + ", x = " + x);



n = 1;
x = 0;
do {
    n++;
    x+=n;
} while (n < 1);
console.log("n = " + n + ", x = " + x);
```

What is the output?

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Repetition Control Structures
# `for`



- The **`for`** statement allows us to have a loop that executes a code block repeatedly until the condition is evaluated to be false
- Syntax:

```
for ([initialization]; [condition]; [final-expression]){
    statement
}
```

```
for (var i = 0; i < 10; i++) {
   console.log("i = " + i);
}
```

What is the output?

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Repetition Control Structures
## *Label*



- The **label** can be used as identifier of a code block that you can refer to
- The label statement can be used with break and continue statement

- Syntax:

```
Label:
    statement
```

```
var i;
myForLoop:
for(i=0; i<10; i++){
    if(i%3==0)
        continue myForLoop;
    console.log(i);
}
```

What is the output?

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Label with **`break`** and **`continue`**



- The **`break`** statement allows us to terminate the current loop, `switch`, or label.
- Syntax:

```
break [label];
```

- The **`continue`** statement allows us to terminate execution of the current iteration of the current or labeled loop and continues execution of the loop with the next iteration.
- Syntax:

```
continue [label];
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Label with **break** and **continue**



```
var i, ind = 5;
myForLoop:
for(i=0; i<10; i++){
        if(i== ind){
            break myForLoop;
        }
}
console.log("Break at i="+ind+"!!");
```

What is the output?

```
var i;
myForLoop:
for(i=0; i<10; i++){
        if(i%3==0)
            continue myForLoop;
        console.log(i);
}
```

What is the output?

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Counter-controlled loops



- **Counter-controlled loops** are loops that are controlled by a **counter** that is updated on every repetition.
- Loops are terminated when a **counter** reaches the given counter value
- Normally, we use counter to control **for** and **while** loops

```
for (var i=5; i>0; i--){
    console.log("i = " + i);
}
```

```
var i = 5;
while(i > 0) {
    console.log("i = " + i);
        i--;
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Sentinel-controlled loops



- **Sentinel-controlled loops** are loops that are terminated, when a sentinel value occurs
- The sentinel value cannot be **predicted** before the loop is entered. Therefore, the number of required loop is **unknown.**
- Normally, we use **`while`** loop to implement sentinel-controlled loops.

```
var flag=0;
while(!flag){
      // if some condition met
      //    set flag = 1;
      // else
      //    keep doing things
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Nesting control structures



Each control structure can be nested where one inside another to any depth.

```
var a,b;
myOuterLoop:
for(a=0;a<10;a++){
    myInnerLoop:
                for(b=0; b<10; b++){
                        if(a%6!=0)
                        continue myOuterLoop;
                for(b=0; b<10; b++){
                        if(a%6!=0)
                        continue myOuterLoop;
                console.log("a=" + a + " b=" + b);
        }
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# Repetition with Array & User-defined object



Array

```
var instructors = ['Wudhichart', 'Wuddy', 'Wud'];
for (var i=0; i < instructors.length; i++) {
    console.log("Instructor" + (i+1) + ": "+instructors[i]);
}
```

User-defined object

```
var wudhichart = {
    firstName: "Wudhichart",
    lastName: "sawangphol",
    age: 30,
    university: "Mahidol",
    feesPaid: true
};

for (var prop in wudhichart) {
    console.log(prop + "=" + wudhichart[prop]);
}
```

COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.

# References


- W3Schools. (2025). *JavaScript Tutorial*. https://www.w3schools.com/js/
- MDN. (2025). *JavaScript*. https://developer.mozilla.org/en-US/docs/Web/JavaScript


COPYRIGHT © 2017-2026 FACULTY OF ICT, MAHIDOL UNIVERSITY. All Rights Reserved.